

PROJECT REPORT

Business Intelligence and Data Visualization

CSL



Faculty name: Dr. Poonam Chaudhry

Student name: Garvit, Gaurav

Roll No.: 21csu322 , 21csu316

Section: DS-V-B

Submission Date: 30-11-2023

Department of Computer Science and Engineering

NorthCap University, Gurugram- 122001, India

Session 2023-24

CONTENTS		
S.N.	Details	Page No.
1	Introduction	
2	Models	
3	Model Experiment	
4	Screenshots	
5	Observation	
6	References	

Project Title:

Spam Email Classification



A spam classifier is a machine learning model designed to identify and categorize incoming messages as either spam or non-spam (ham). It analyzes features of messages, such as content, sender information, and formatting, to distinguish between legitimate and unwanted communications. Spam classifiers are commonly used in email systems, messaging platforms, and comment sections to automatically filter out or flag potential spam, providing users with a more efficient and secure communication experience.

AI and ML are employed in spam classification by training models on labeled datasets to recognize patterns indicative of spam. These models use features like text content and sender behavior to autonomously identify and filter out spam messages, enhancing the accuracy and efficiency of spam detection systems.

About The Project:

SMS / Mail Classifier

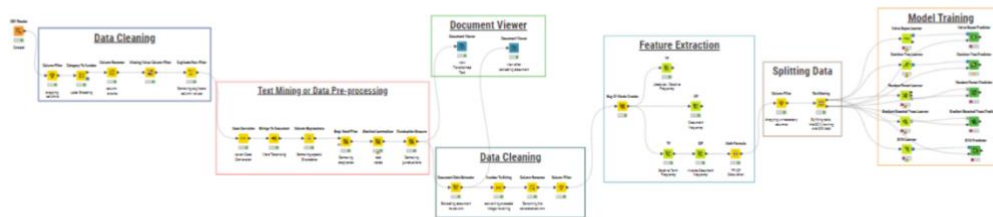
The "SMS/Mail Spam Detection" project leverages AI and ML techniques, including Natural Language Processing (NLP) and the Natural Language Toolkit (NLTK), to develop a robust system for identifying and filtering out spam messages in SMS and email communications. By training the model on labeled datasets, it learns to recognize patterns indicative of spam, considering factors like text content, sender behavior, and linguistic cues. The implementation of NLP and NLTK enhances the model's understanding of the context and nuances within messages.



This project offers a valuable solution to the pervasive issue of unwanted communication, contributing to a cleaner and more secure digital environment. Efficient spam detection not

only saves users from the inconvenience of sorting through irrelevant messages but also protects against potential scams and phishing attacks. The societal impact lies in fostering improved communication experiences, safeguarding users from malicious activities, and promoting trust in digital interactions.

WORKFLOW SCREENSHOT :



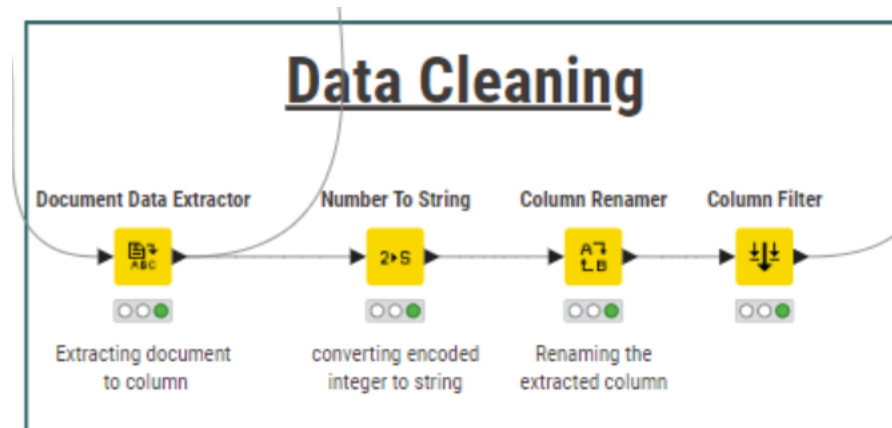
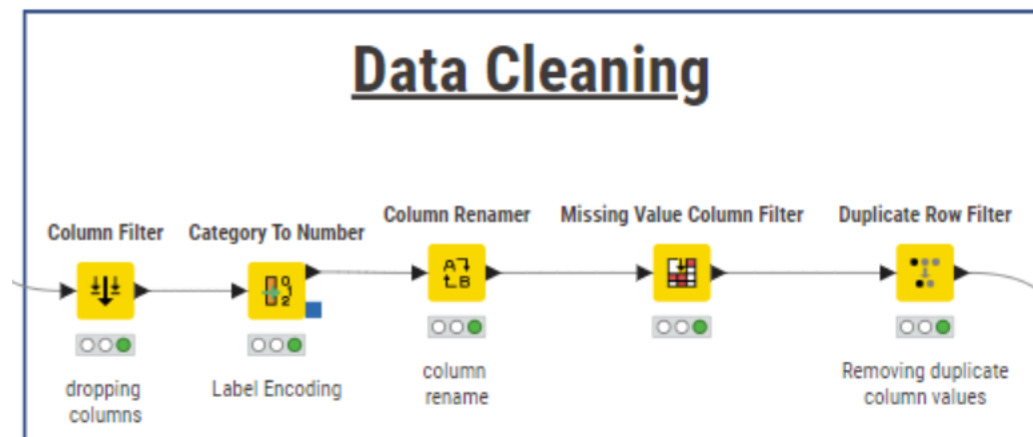
Data Cleaning



The provided code is related to data preprocessing and exploration for a DataFrame, likely associated with a machine learning or data analysis project.

This code is essential for a viewer as it prepares the dataset for further analysis or machine learning tasks by handling missing values, dropping unnecessary columns, renaming columns for clarity, converting categorical labels into numerical format, and ensuring data quality by removing duplicates. It provides a clean and organized dataset, ready for subsequent stages in the data science workflow.

DATA CLEANING CODE SCREENSHOT :



Data Processing & Visualization



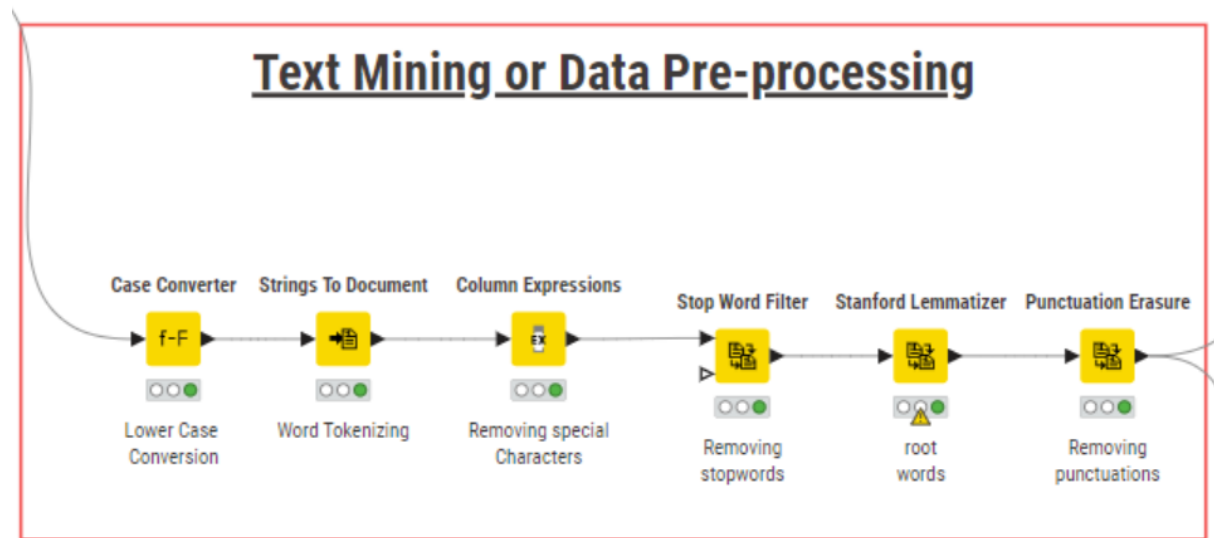
The provided code performs text preprocessing and analysis for spam and ham (non-spam) messages. It goes through several essential steps to clean and transform the text data:

1. Lowercasing: Converts all text to lowercase for uniformity.
2. Tokenization: Splits the text into individual words or tokens.
3. Removing Special Characters: Eliminates non-alphanumeric characters from the tokens.
4. Removing Stop Words and Punctuation: Filters out common English stop words and punctuation marks.
5. Stemming: Reduces words to their root or base form using the Porter stemming algorithm.

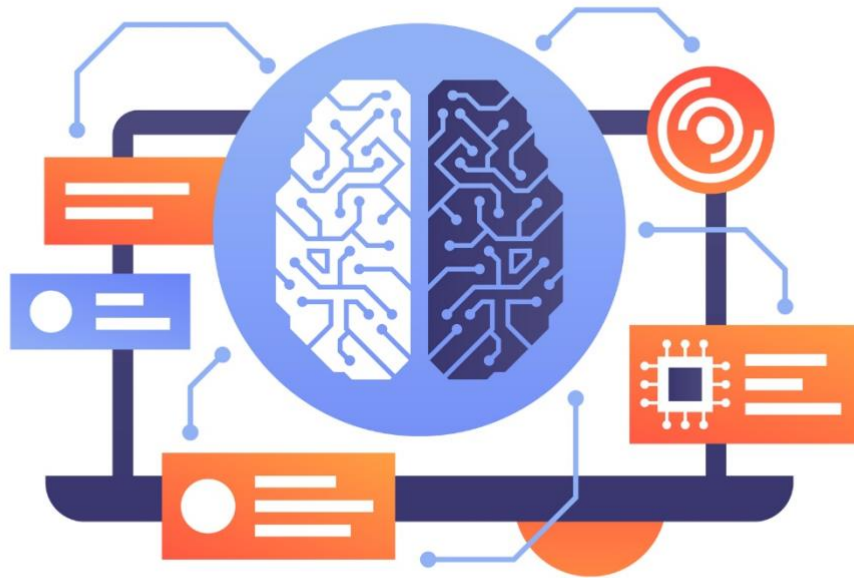
After preprocessing, the code generates word clouds for both spam and ham messages, visually representing the most frequent words. Additionally, it creates bar charts displaying the top 30 words for both spam and ham messages.

The necessity of this code for a viewer lies in its ability to enhance understanding and visualization of the most common words in spam and non-spam messages. This analysis can aid in developing more effective spam detection models and contribute to insights about the characteristics of spam and ham messages.

DATA PROCESSING & VISUALIZATION SCREENSHOT :



Machine learning model Building



Here's an explanation of each model and the potential reasons for using them in the context of SMS / Mail Spam Classifier :

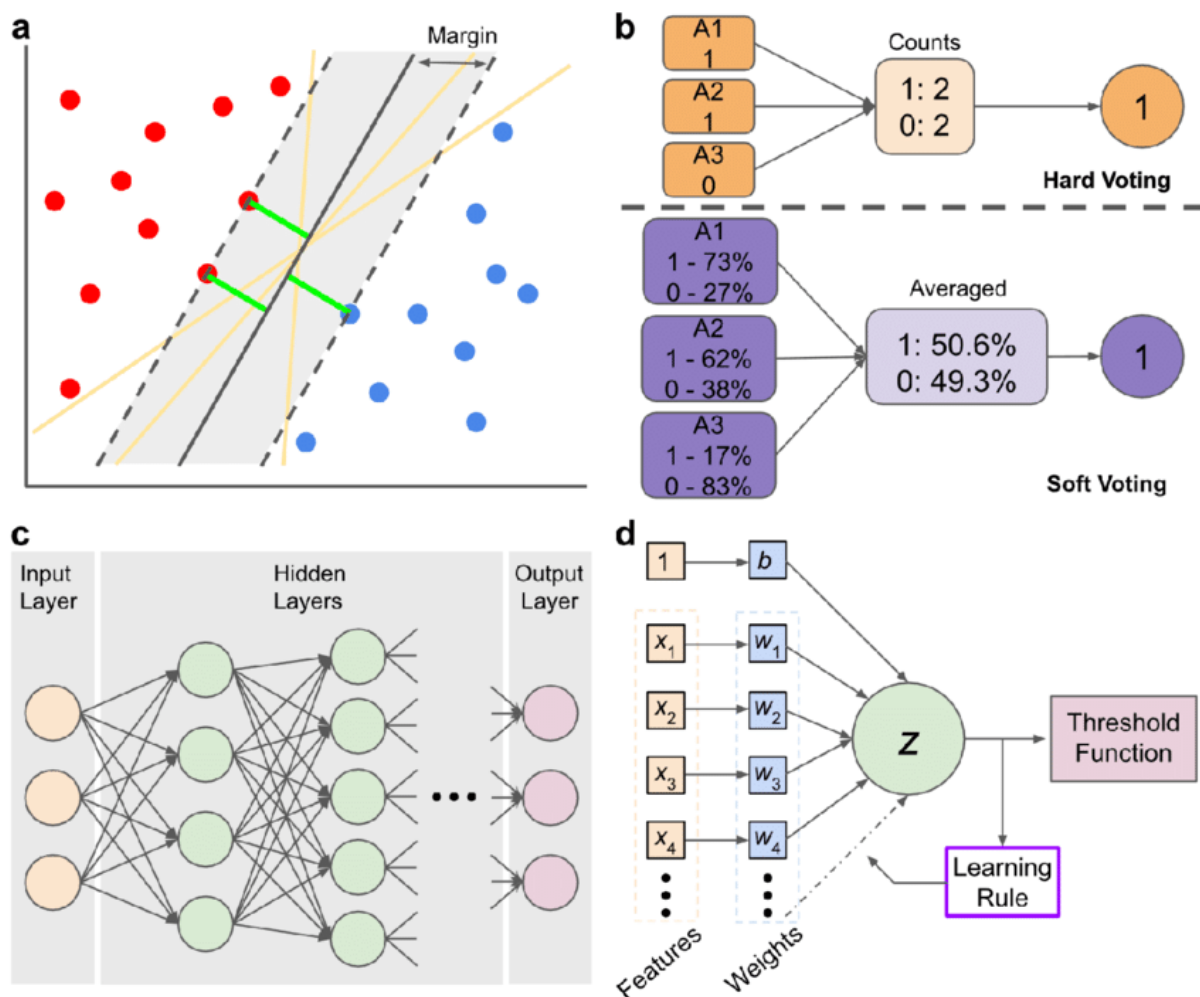
1. Support Vector Classifier (SVC):

Type : Non-Linear

Use Case : Classification of both linear and non-linear

Working Principle : Finds the hyperplane that best separates data into classes

Key Features : Effective in high-dimensional spaces, versatile with different kernels



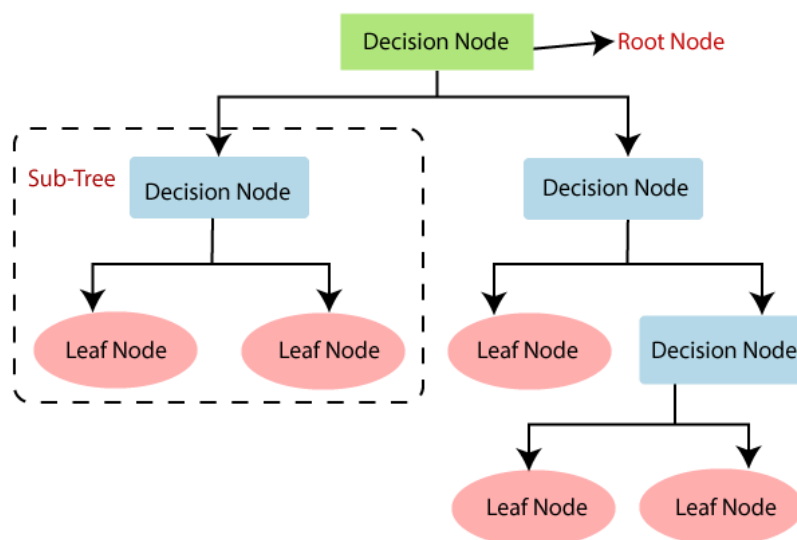
2. Decision Tree:

Type : Non-Linear

Use Case : Classification, Regression

Working Principle : Makes decisions by recursively splitting data based on feature conditions

Key Features : Intuitive, easy to understand, handles both numerical and categorical data



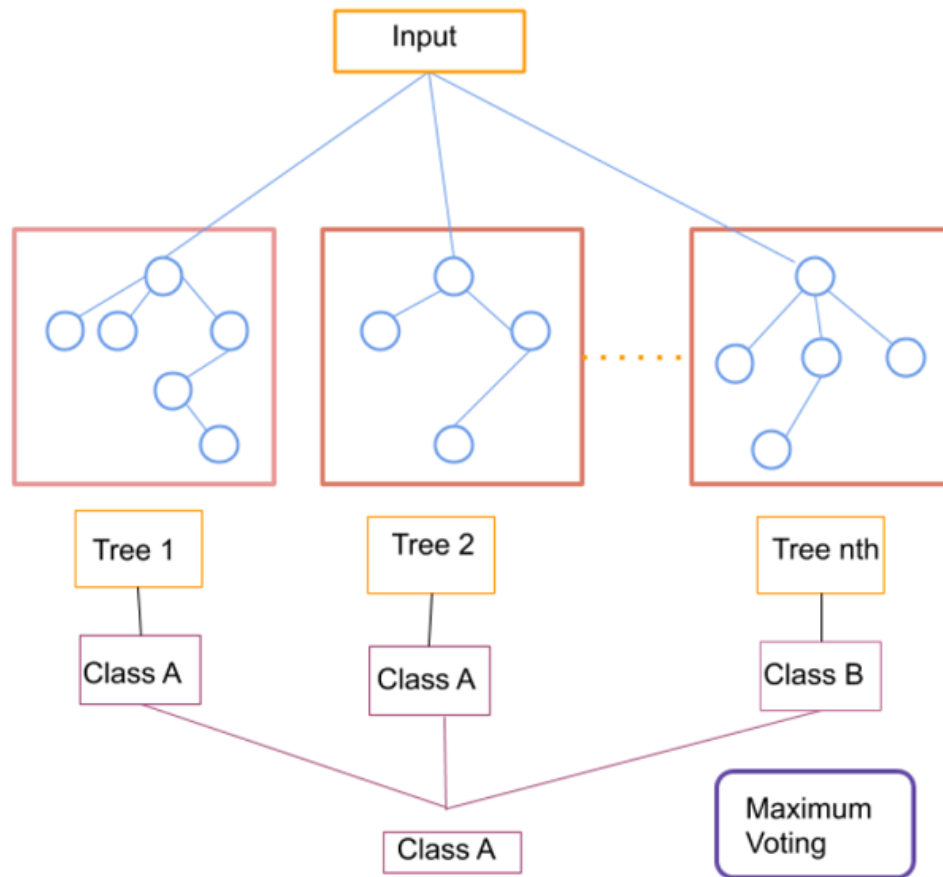
3. Random Forest:

Type : Ensemble (Bagging)

Use Case : Classification, Regression

Working Principle : Constructs multiple decision trees and combines their predictions

Key Features : Robust, reduces overfitting, handles large datasets



4. Gradient Boosting:

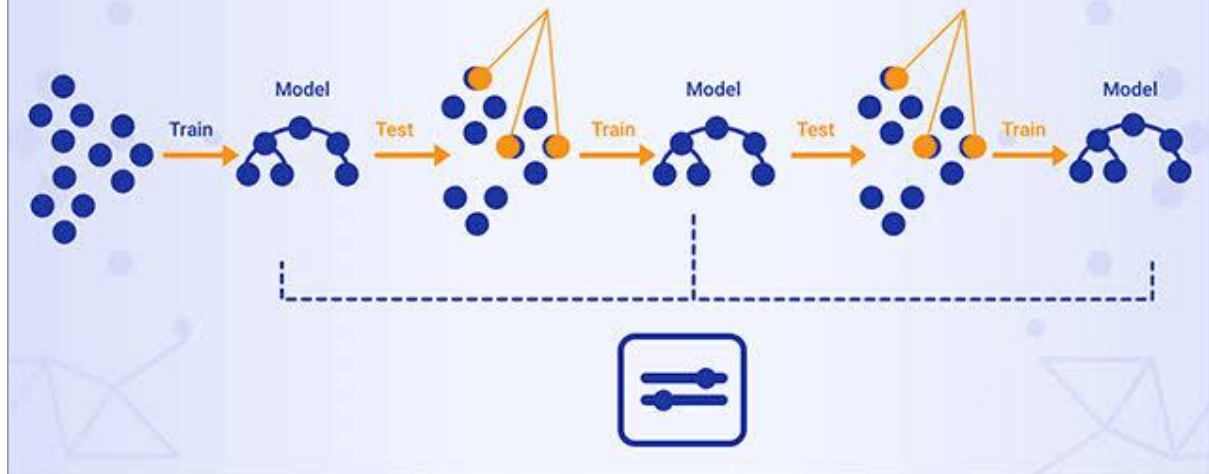
Type : Ensemble (Boosting)

Use Case : Regression, Classification

Working Principle : Builds weak models sequentially, each correcting errors of the previous one

Key Features : High accuracy, handles complex relationships

Gradient Boosting



Naive Bayes:

- **Type:** Probabilistic, Generative, Discriminative
- **Specific Types:** Multinomial Naive Bayes, Gaussian Naive Bayes, Bernoulli Naive Bayes
- **Use Case:** Classification tasks, such as spam detection, sentiment analysis, document classification, and more.
- **Working Principle:** Naive Bayes is based on Bayes' theorem, which calculates the probability of a hypothesis given the observed evidence. The "naive" assumption is that features are conditionally independent given the class label. Despite this simplifying assumption, Naive Bayes often performs well in practice and is computationally efficient.

Key Features:

1. Simplicity and Speed:

- Naive Bayes is computationally efficient and easy to implement.

2. Probabilistic Framework:

- Provides a probabilistic framework for making predictions, allowing estimation of the likelihood of different outcomes.

3. Assumption of Independence:

- The "naive" assumption of feature independence simplifies calculations and makes it particularly suited for high-dimensional data.

4. Effective for Text Classification:

- Particularly effective in text classification tasks, such as spam filtering and sentiment analysis.

5. Robust with Small Datasets:

- Works well even with small datasets and is less prone to overfitting.

6. Online Learning:

- Naive Bayes supports online learning, where the model can be continuously updated as new data becomes available.

7. Handles Categorical Data:

- Suitable for both numerical and categorical features, making it versatile in handling different types of data.

8. Interpretability:

- The model's probabilistic nature allows for easy interpretation of predictions and the factors influencing them.

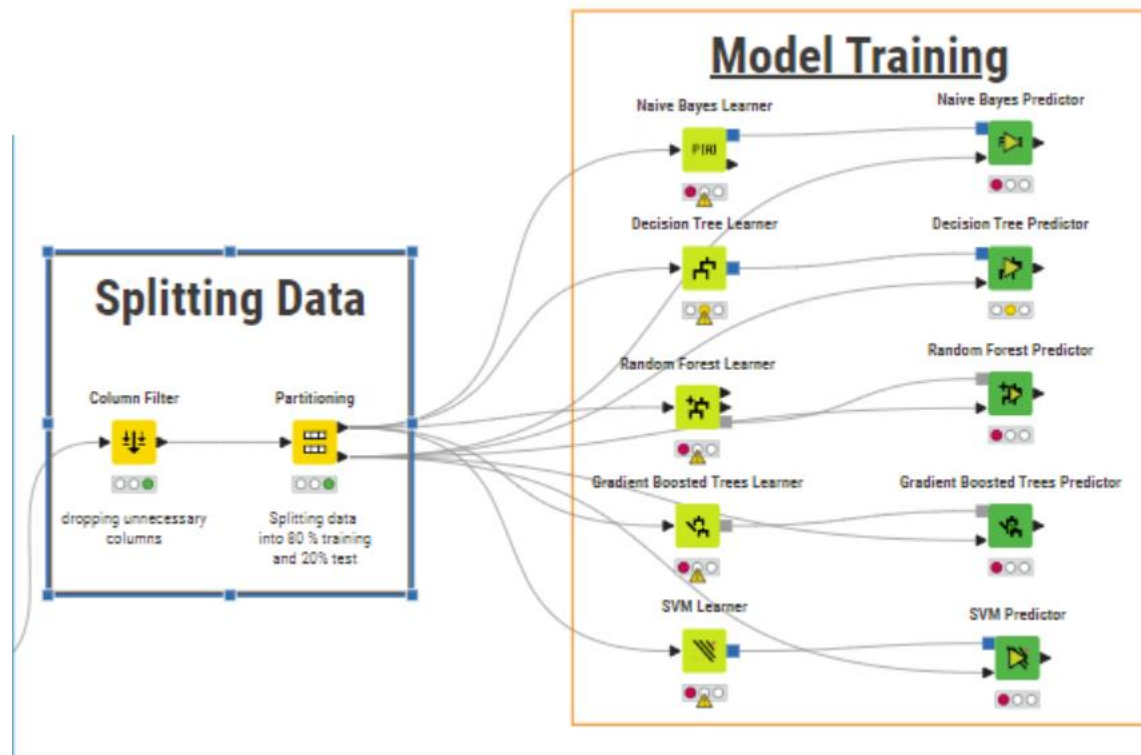
9. Adaptability:

- Can handle missing data gracefully, making it adaptable to real-world datasets with incomplete information.

10. Scalability:

- Scales well to large datasets, making it applicable in various domains.

MODEL BUILDING SCREENSHOT :



Feature Extraction

I've used Bag of words (CountVectorizer) first for model building, and for model improvement I've used TfidfVectorizer with the max feature 3000.

Both CountVectorizer (Bag of Words) and TF-IDF (Term Frequency-Inverse Document Frequency) vectorizers are techniques used in natural language processing to convert text data into numerical vectors. The key differences lie in how they assign weights to words.

1. CountVectorizer / Bag of Words:

Representation: It represents each document as a vector of word counts.

Weighting: The importance of a word is determined by its frequency in the document.

2. TF-IDF Vectorizer:

Representation: It represents each document as a vector, where each word is assigned a weight based on its TF-IDF score.

Weighting: TF-IDF considers both the frequency of a word in a document and its rarity across all documents in the corpus.

Differences:

Weighting Mechanism: While CountVectorizer uses the raw frequency of words, TF-IDF considers both the term frequency (TF) and the inverse document frequency (IDF). TF-IDF down-weights common words that appear across many documents and up-weights words that are important within a specific document.

Normalization: TF-IDF normalizes the vectors, making them less sensitive to document length compared to CountVectorizer. This can be important when comparing documents of varying lengths.

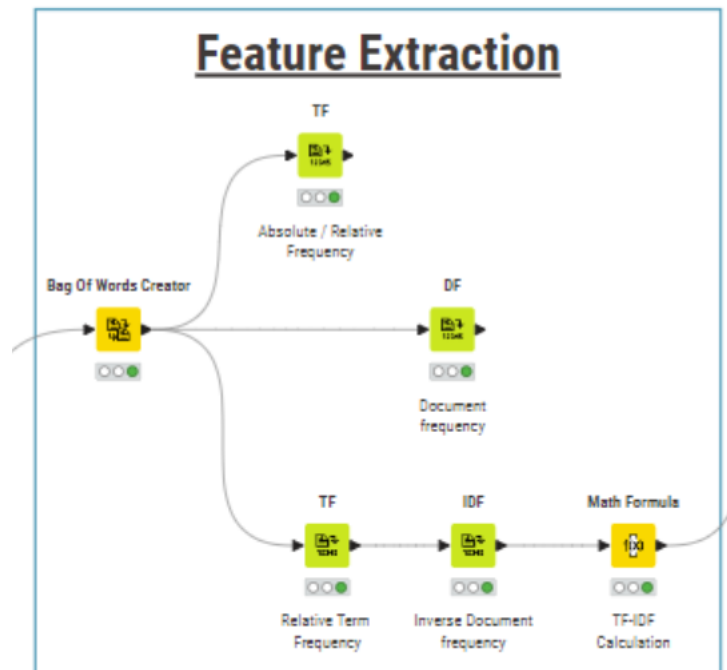
Handling Common Words: CountVectorizer may give higher importance to common words, such as "the" or "and," which may not carry much meaningful information. TF-IDF tends to downplay the importance of such common words.

Performance: In many cases, TF-IDF performs better than CountVectorizer because it provides a more nuanced representation of the importance of words in a document. It helps capture the uniqueness of words and their significance in a specific context.

Max Features: Setting `max\_features` to limit the number of features can be beneficial for both vectorizers, especially when dealing with high-dimensional data. It helps control the dimensionality of the feature space.

In summary, TF-IDF is often preferred over CountVectorizer when the goal is to capture the significance of words in a document while downplaying the influence of common words. The choice between them depends on the specific characteristics of the text data and the objectives of the analysis.

MODEL IMPROVEMENT SCREENSHOTS :



Reasons for Using Multiple Models:

Diversity in Approaches:

Each model makes different assumptions and uses distinct mechanisms for classification. Using multiple models provides a diverse set of approaches to capturing patterns in the data.

Ensemble Learning:

Combining the predictions of multiple models can result in a more robust and accurate overall model. Ensemble methods like Random Forest naturally leverage this principle.

Model Comparison:

By implementing different models, you can compare their performances using metrics like accuracy, precision, recall, and F1-score. This helps identify the most suitable model for your specific SMS / Mail spam classification task.

Handling Complex Data:

SMS and mail spam data exhibit inherent variability, and various models can excel in capturing diverse facets of this complexity. Incorporating multiple models enables a nuanced exploration of spam content, considering its multifaceted nature. To elaborate, the incorporation of models such as Logistic Regression, Decision Trees, and K-Neighbors in your SMS/mail spam classification project demonstrates a strategic approach to model diversity. Each model, with its unique strengths, contributes to a comprehensive strategy for effectively tackling the intricate challenges involved in identifying and categorizing spam messages. The ultimate selection among these models may hinge on empirical performance evaluations and specific considerations tailored to the nuances of your spam classification project.

Nodes Used In Project:

Certainly! Here's a brief explanation and a use case for each of the mentioned nodes in KNIME:

1. CSV Reader:

- **Definition:** Reads data from a CSV file.
- **Use Case:** Importing a dataset stored in a CSV file for further analysis in KNIME.

2. Column Filter:

- **Definition:** Filters out unwanted columns from the dataset.
- **Use Case:** Selecting only relevant features for analysis and removing unnecessary columns.

3. Column Renamer:

- **Definition:** Renames columns in the dataset.
- **Use Case:** Standardizing column names or making them more descriptive.

4. Missing Value Column Filter:

- **Definition:** Filters rows based on the presence or absence of missing values.
- **Use Case:** Removing or handling rows with missing data in the dataset.

5. Duplicate Row Filter:

- **Definition:** Filters out duplicate rows in the dataset.
- **Use Case:** Identifying and removing duplicate records from a dataset.

6. Case Converter:

- **Definition:** Converts the case of string data (e.g., to uppercase or lowercase).
- **Use Case:** Standardizing the case of text data for consistency.

7. String to Document:

- **Definition:** Converts text data into document format for text mining.
- **Use Case:** Preprocessing text data for natural language processing tasks.

8. Column Expression:

- **Definition:** Applies expressions to create new columns or modify existing ones.
- **Use Case:** Calculating new features or transforming existing columns using mathematical expressions.

9. Stop Word Filter:

- **Definition:** Filters out common words (stop words) from text data.
- **Use Case:** Preprocessing text data in natural language processing to remove common, uninformative words.

10. Stanford Lemmatizer:

- **Definition:** Applies lemmatization to text data.
- **Use Case:** Preprocessing text data to reduce words to their base or root form.

11. Punctuation Erasure:

- **Definition:** Removes punctuation from text data.
- **Use Case:** Cleaning text data by removing punctuation marks.

12. Document Viewer:

- **Definition:** Visualizes document data.
- **Use Case:** Inspecting the content of documents in a workflow.

13. Document Data Extractor:

- **Definition:** Extracts information from document data.
- **Use Case:** Extracting relevant features or metadata from document-type data.

14. Number to String:

- **Definition:** Converts numerical data to string format.
- **Use Case:** Preparing data for downstream processing that requires string data.

15. Bag of Words Creator:

- **Definition:** Creates a bag-of-words representation from text data.
- **Use Case:** Text mining and natural language processing tasks that involve word frequency analysis.

16. TF (Term Frequency):

- **Definition:** Calculates term frequency in text data.
- **Use Case:** Preprocessing for text mining, where the frequency of terms is important.

17. DF (Document Frequency):

- **Definition:** Calculates document frequency in text data.
- **Use Case:** Analyzing the occurrence of terms across multiple documents.

18. IDF (Inverse Document Frequency):

- **Definition:** Calculates inverse document frequency in text data.
- **Use Case:** Weighing the importance of terms in a collection of documents.

19. Math Formula:

- **Definition:** Applies mathematical formulas to columns.
- **Use Case:** Creating new calculated columns based on mathematical operations.

20. Partitioning:

- **Definition:** Splits the dataset into training and testing sets.
- **Use Case:** Preparing data for machine learning by creating separate sets for training and evaluation.

21. Naive Bayes Learner:

- **Definition:** Trains a Naive Bayes classification model.
- **Use Case:** Building a predictive model for classification tasks.

22. Decision Tree Learner:

- **Definition:** Trains a decision tree classification model.
- **Use Case:** Building a decision tree model for classification tasks.

23. Random Forest Learner:

- **Definition:** Trains a random forest classification model.
- **Use Case:** Building an ensemble model for classification tasks.

24. Gradient Boosting Trees Learner:

- **Definition:** Trains a gradient boosting trees classification model.
- **Use Case:** Building a boosted ensemble model for classification tasks.

25. SVM Learner:

- **Definition:** Trains a support vector machine classification model.
- **Use Case:** Building a SVM model for classification tasks.

26. Naive Bayes Predictor:

- **Definition:** Applies a trained Naive Bayes model to make predictions.
- **Use Case:** Using a Naive Bayes model to predict class labels on new data.

27. Decision Tree Predictor:

- **Definition:** Applies a trained decision tree model to make predictions.
- **Use Case:** Using a decision tree model to predict class labels on new data.

28. Random Forest Predictor:

- **Definition:** Applies a trained random forest model to make predictions.
- **Use Case:** Using a random forest model to predict class labels on new data.

29. Gradient Boosting Predictor:

- **Definition:** Applies a trained gradient boosting model to make predictions.
- **Use Case:** Using a gradient boosting model to predict class labels on new data.

30. SVM Predictor:

- **Definition:** Applies a trained support vector machine model to make predictions.
- **Use Case:** Using an SVM model to predict class labels on new data.

31. Scorer:

- **Definition:** Evaluates the performance of a predictive model.

- Use Case: Assessing the accuracy, precision, recall, and other metrics of a classification model.

Each of these nodes serves a specific purpose in the KNIME Analytics Platform and contributes to the data preprocessing, transformation, and machine learning stages of a workflow. The combination of these nodes allows users to construct powerful and flexible workflows for a variety of data analytics tasks.

Dataset Overview:

The dataset consists of the following columns:

1. v1 : This column has entries in the form of ham and spam
2. v2 : This column has the messages or text written

Dataset Characteristics:

- The dataset is free of any null values in any row, ensuring completeness for analysis.

Interpretation and Context:

- The dataset appears to be related to text classification, potentially for SMS / Mail spam detection.
- The 'v2' column likely provides categorical labels for the type of content present in the text.

Recommendations for Analysis:

- Consider using the 'v2' column as the input feature and the 'v1' column as the target variable for text classification tasks.
- Explore the distribution of samples across different classes in the 'v2' column.
- Preprocess the 'v2' column, which may involve text cleaning and tokenization, before building machine learning models.

Observation :

The data was imbalanced, therefore precision mattered more than the accuracy.

1. Feature Representation: The use of TF-IDF implies that the model is considering not only term frequency (TF) but also the importance of terms across the entire dataset (inverse document frequency, IDF). This may be beneficial for capturing the uniqueness of terms in spam messages.

2. Dimensionality Control: Setting the maximum features to 3000 suggests an effort to control the dimensionality of the feature space. This is a prudent step, especially in text classification tasks where the vocabulary can be extensive. It helps prevent overfitting and improves model efficiency.

3. Naive Bayes Suitability: Multinomial Naive Bayes is well-suited for text classification tasks, particularly when dealing with discrete features like word counts. It assumes that features are multinomially distributed, making it appropriate for tasks involving word frequencies in documents.

4. Performance and Generalization: The model's performance, especially when evaluated on a testing dataset, should be considered. High accuracy on both training and testing data indicates good generalization, while a significant gap might suggest overfitting.

5. Interpretability: Naive Bayes models, including Multinomial Naive Bayes, are known for their simplicity and interpretability. This makes them suitable for scenarios where understanding the decision-making process is crucial.

7. Model Comparison: While Naive Bayes is performing well in your observation, it's also valuable to compare its performance with other models. Experimenting with different classifiers and vectorizers can provide a more comprehensive understanding of the dataset.

8. Data Characteristics: The success of the chosen model and vectorizer could be influenced by the characteristics of your SMS/Mail spam dataset. The diversity and nature of spam messages play a crucial role in determining which model performs optimally.

In conclusion, your observation suggests a well-thought-out approach to SMS/Mail spam detection, emphasizing the effectiveness of Multinomial Naive Bayes with TF-IDF vectorization and feature control. Further analysis, including performance metrics and potential comparisons with other models, will enrich your understanding of the model's capabilities and limitations.